

Wahrscheinlichkeitstheorie und Statistik (Stochastik)

Lineare Kongruenz-Generatoren

Dr. rer. nat. Johannes Riesterer

Pseudozufallszahlen

Motivation

Problem

Ein Computer ist deterministisch – echte Zufallszahlen kann er nicht erzeugen. Für Simulationen (Monte Carlo, Bootstrapping, Kryptographie, Spiele, ...) benötigt man jedoch Folgen, die sich wie Zufall verhalten.

Idee: Pseudozufallszahlen

Eine deterministische Folge $X_0, X_1, X_2, \dots$, die statistische Tests auf Zufälligkeit besteht. Vorteile:

- reproduzierbar (Seed),
- schnell berechenbar,
- geringer Speicherbedarf.

Linearer Kongruenz-Generator (LCG)

Definition

Rekursionsvorschrift

Wähle

- Modulus $m \in \mathbb{N}$, $m > 0$,
- Multiplikator $a \in \{1, \dots, m-1\}$,
- Inkrement $c \in \{0, \dots, m-1\}$,
- Startwert (Seed) $X_0 \in \{0, \dots, m-1\}$.

Die Folge ist definiert durch

$$X_{n+1} = (a \cdot X_n + c) \bmod m.$$

Normierung auf $[0, 1)$

$$U_n = \frac{X_n}{m} \in [0, 1).$$

Beispiel

$$m = 9, a = 7, c = 4, X_0 = 3$$

Berechnung von Hand

n	Rechnung	X_n
0	—	3
1	$(7 \cdot 3 + 4) \bmod 9 = 25 \bmod 9$	7
2	$(7 \cdot 7 + 4) \bmod 9 = 53 \bmod 9$	8
3	$(7 \cdot 8 + 4) \bmod 9 = 60 \bmod 9$	6
4	$(7 \cdot 6 + 4) \bmod 9 = 46 \bmod 9$	1
5	$(7 \cdot 1 + 4) \bmod 9 = 11 \bmod 9$	2
6	$(7 \cdot 2 + 4) \bmod 9 = 18 \bmod 9$	0
7	$(7 \cdot 0 + 4) \bmod 9 = 4 \bmod 9$	4
8	$(7 \cdot 4 + 4) \bmod 9 = 32 \bmod 9$	5
9	$(7 \cdot 5 + 4) \bmod 9 = 39 \bmod 9$	3

Nach $9 = m$ Schritten wiederholt sich die Folge – maximale Periode erreicht.

Periode

Hull–Dobell-Bedingungen

Beobachtung

Da $X_n \in \{0, 1, \dots, m-1\}$, gibt es höchstens m verschiedene Werte. Spätestens nach m Schritten wiederholt sich die Folge.

Satz (Hull & Dobell, 1962)

Ein LCG hat genau dann die maximale Periode m , wenn:

- 1 $\gcd(c, m) = 1$,
- 2 für jeden Primfaktor p von m gilt $p \mid (a - 1)$,
- 3 ist $4 \mid m$, so auch $4 \mid (a - 1)$.

Beispiele aus der Praxis

	m	a	c
glibc rand()	2^{31}	1103515245	12345
Numerical Recipes	2^{32}	1664525	1013904223

```
def lcg(seed, a=1664525, c=1013904223, m=2**32, n=10):  
    x = seed  
    for _ in range(n):  
        x = (a * x + c) % m  
        yield x / m      # in [0, 1)  
  
print(list(lcg(seed=42, n=5)))
```

Bemerkung

Mit demselben Seed liefert der Generator stets dieselbe Folge – unverzichtbar für reproduzierbare Simulationen.

Eigenschaften und Grenzen

Vorteile

- sehr schnell, eine Multiplikation und eine Addition pro Schritt,
- minimaler Speicherbedarf (ein Integer),
- deterministisch reproduzierbar.

Nachteile

- **Marsaglia-Effekt:** Die Tupel $(X_n, X_{n+1}, \dots, X_{n+k-1})$ liegen in $[0, 1)^k$ auf wenigen Hyperebenen $\Rightarrow$ schlechte Verteilung in hohen Dimensionen.
- niederwertige Bits haben oft sehr kurze Perioden,
- **nicht kryptographisch sicher:** aus wenigen Ausgaben lassen sich a, c, m rekonstruieren.

Heutige Standards

Mersenne Twister (`numpy.random`, `Python random`), PCG, xorshift,

...